

OOP

INTRODUCTION TO UML

“Code shows how a system is built. A model helps us decide what should be built.”

BEFORE WE WRITE THE CLASS

Imagine that you are developing a music application. You know that the system will contain songs, playlists, albums, and users. You can begin writing C++ immediately—but first you need to answer simpler design questions:

- What information belongs to each class?
- What operations should each class provide?
- How are the classes connected?

A UML diagram gives us a compact visual language for discussing these decisions before the details of the code hide the design.

LEARNING OBJECTIVES

- Explain what UML is and why software developers use models.
- Recognize a UML class diagram and its three compartments.
- Write attributes and operations using standard UML notation.
- Interpret public, private, and protected visibility symbols.
- Translate a simple C++ class declaration into a UML class diagram and back.
- Read multiplicities on an association between two classes.
- Distinguish association, aggregation, composition, and inheritance at an introductory level.
- Distinguish a class diagram from an object diagram.

BEFORE WE BEGIN

You should already understand struct, functions, parameters, classes, objects, public and private access, and the basic idea of encapsulation.

Important: UML is a design and communication tool. It does not replace code, and a diagram does not need to show every implementation detail.

WHAT IS UML?

UML stands for Unified Modeling Language. It is a standardized visual language for describing software systems.

UML includes several kinds of diagrams. Some describe structure, some describe behavior, and some describe interactions over time. In this course, we begin with the class diagram because it connects directly to the classes we write in C++.

A MODEL IS A SIMPLIFIED VIEW

A map does not show every tree, window, or stone in a city. It shows the information needed for a particular purpose. A software model works in the same way.

- A high-level diagram may show only class names and relationships.
- A design diagram may also show attributes and operations.
- An implementation diagram may include types, parameters, and visibility.

Key idea: The best diagram is not the diagram with the most detail. It is the diagram that communicates the right detail clearly.

WHY NOT BEGIN WITH CODE?

Consider a team discussing a music application. One developer imagines that Playlist owns Song objects. Another imagines that the same Song can appear in many playlists. Both can write valid C++, but they are designing different systems.

A diagram makes the decision visible. It helps the team discuss structure before implementation choices become expensive to change.

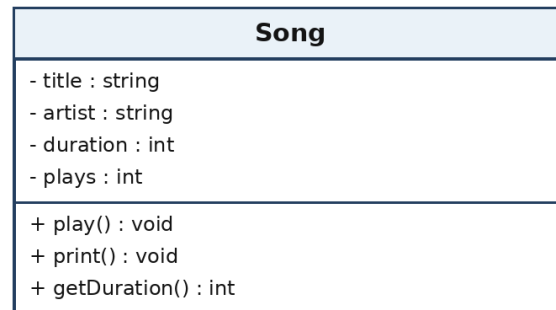
UML HELPS US

- see the main classes at one glance,
- divide responsibilities between classes,
- discover missing or unnecessary classes,
- communicate a design without reading every source file,
- keep design discussions independent of a specific programming language.

The diagram and the code should tell the same design story, even though their syntax is different.

THE UML CLASS DIAGRAM

A class is drawn as a rectangle divided into up to three compartments:



A UML class with a name, attributes, and operations

- Top compartment — the class name.
- Middle compartment — attributes: the data stored by the class.
- Bottom compartment — operations: the behavior provided by the class.

The class name is normally centered and bold. Attributes and operations are written one per line.

Do not confuse the model with the syntax: UML writes the name before the type; C++ writes the type before the name.

WRITING ATTRIBUTES

The general UML notation for an attribute is:

```
visibility name : type
```

For example:

- title : string
- duration : int
- plays : int

The hyphen means private. The colon separates the attribute name from its type.

UML AND C++ USE OPPOSITE ORDER

// UML	// C++
- duration : int	int duration;
- title : string	string title;

Remember: UML: name : type. C++: type name.

WRITING OPERATIONS

The general UML notation for an operation is:

```
visibility name(parameter : type) : returnType
```

Examples:

```
+ play() : void  
+ setDuration(seconds : int) : void  
+ getDuration() : int  
+ isLongerThan(other : Song) : bool
```

The plus sign means public. Parameters are written as name followed by type. The return type appears after the closing parenthesis.

EQUIVALENT C++ DECLARATIONS

```
void play();  
void setDuration(int seconds);  
int getDuration() const;  
bool isLongerThan(const Song& other) const;
```

UML can show additional details such as `const`, but introductory diagrams often omit language-specific information unless it matters to the design.

VISIBILITY SYMBOLS

UML uses a symbol at the beginning of an attribute or operation to show who may access it.

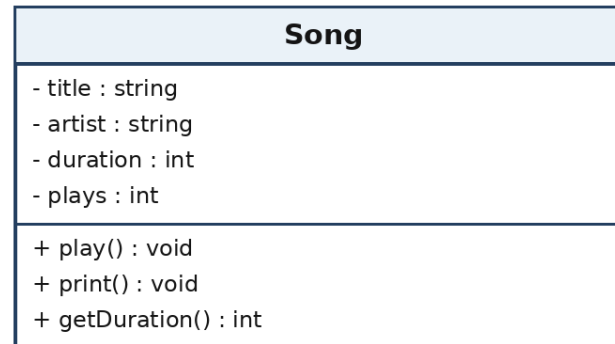
Symbol	Meaning	C++ access specifier
+	public	public:
–	private	private:
#	protected	protected:

In an encapsulated class, data members are usually private and the controlled interface is public.

```
class Song
{
private:
    string title;
    int duration;

public:
    void play();
    int getDuration() const;
};
```


A COMPLETE SONG DIAGRAM



We can read the diagram from top to bottom:

- The class is named Song.
- A Song stores a title, artist, duration, and play count.
- All four attributes are private.
- Clients may call `play()`, `print()`, and `getDuration()`.
- `getDuration()` returns an int; the other operations return void.

Notice what the diagram does not show: function bodies, validation conditions, include directives, memory layout, or the exact output format of `print()`. Those belong in the implementation or in a more detailed specification.

FROM UML TO C++

The class diagram describes the interface and state. The C++ declaration expresses the same decisions using language syntax.

```
#include <string>
using std::string;

class Song
{
private:
    string title;
    string artist;
    int duration;
    int plays;

public:
    void play();
    void print() const;
    int getDuration() const;
};
```

The diagram is quicker to scan. The declaration is precise enough for the compiler. In software design we often use both.

Design check: Every important diagrammed attribute or operation should have a clear counterpart in the class declaration, unless the diagram intentionally shows a higher level of abstraction.

RELATIONSHIPS BETWEEN CLASSES

Real systems rarely contain isolated classes. UML uses different line endings to express different kinds of relationships.

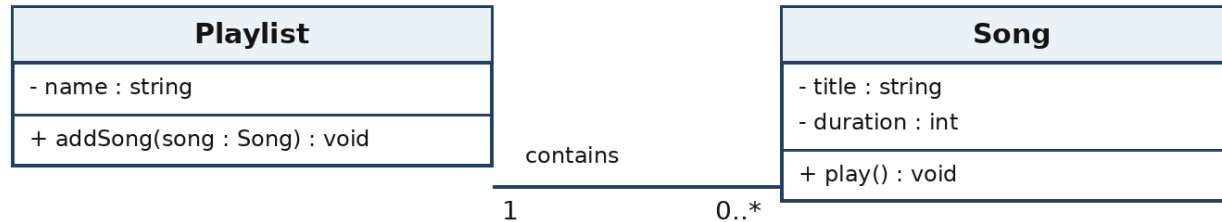
- Association — objects know about or work with one another.
- Aggregation — a weak whole–part relationship; the part can exist independently.
- Composition — a strong whole–part relationship; the whole controls the lifetime of the part.
- Inheritance — one class is a specialized kind of another class.

These symbols are not decoration. Each one makes a claim about the design. Use the simplest relationship that accurately describes the system.

At this stage: You should be able to read these relationships. Their complete C++ implementation will be developed in later units.

ASSOCIATION AND MULTIPLICITY

A plain line represents an association. Multiplicity tells us how many objects may participate at each end.



One Playlist may contain zero or more Song objects

- 1 means exactly one.
- 0..1 means zero or one.
- * or 0..* means any number, including none.
- 1..* means one or more.

Read the diagram in both directions: a Playlist is associated with zero or more Songs; each shown relationship belongs to one Playlist.

Multiplicity describes the design. It does not decide whether C++ stores objects, pointers, references, indices, or another representation.

AGGREGATION

Aggregation represents a whole–part relationship in which the part can still exist independently. It is drawn with a hollow diamond at the whole.



The hollow diamond is placed next to Album, the whole

An Album groups Songs, but a Song can exist without that Album. The same Song may also be used elsewhere in the music application.

HOW TO TEST THE RELATIONSHIP

Ask: if the whole object disappears, can the part still meaningfully exist?

- If yes, aggregation may be appropriate.
- If there is no important whole–part meaning, use a simple association.

Be careful: Use a simple association when whole–part ownership is not important. Add a diamond only when it communicates a real design decision.

COMPOSITION

Composition is a strong whole–part relationship. It is drawn with a filled diamond at the whole.



The filled diamond is placed next to MusicPlayer, the whole

In this design, **Settings** is an internal part of one **MusicPlayer**. Its lifetime is controlled by the **MusicPlayer**; it is not shared as an independent domain object.

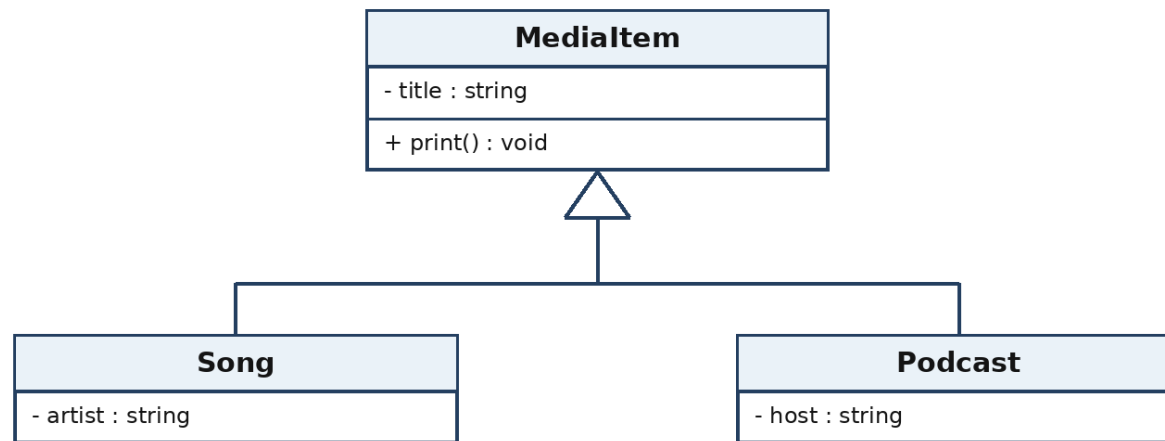
AGGREGATION COMPARED WITH COMPOSITION

- Aggregation: the part can exist independently of the whole.
- Composition: the part belongs exclusively to the whole and shares its lifetime.

The difference is about ownership and lifetime, not simply whether one class contains a field whose type is another class.

INHERITANCE

Inheritance represents an is-a relationship. A solid line with a hollow triangular arrow points toward the more general base class.



Song and Podcast are specialized kinds of Medialtem

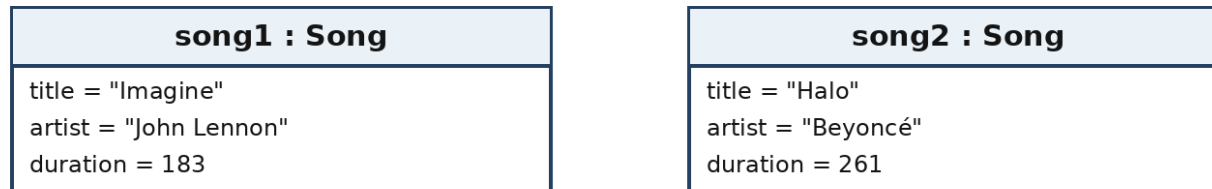
- A Song is a Medialtem.
- A Podcast is a Medialtem.
- Medialtem contains the state and behavior shared by both derived classes.

Do not use inheritance merely because two classes share a few attributes. The relationship should express a genuine conceptual specialization.

Later in the course: We will connect this notation to C++ base classes, derived classes, virtual functions, and polymorphism.

CLASS DIAGRAMS AND OBJECT DIAGRAMS

A class diagram describes types: what every object of a class may contain and do. An object diagram describes particular objects at one moment in time.



Two individual Song objects and their current attribute values

- Song is a class name.
- song1 and song2 are object names.
- The notation song1 : Song means that song1 is an instance of Song.

Object diagrams are useful when tracing a scenario, explaining links between particular objects, or checking whether a class design can represent the required data.

Remember: A class diagram is a blueprint. An object diagram is a snapshot.

FROM A REQUIREMENT TO A DIAGRAM

Suppose the requirement says:

Requirement: A playlist has a name. It can contain any number of songs. A song stores its title, artist, and duration. A song may appear in more than one playlist.

STEP 1 — IDENTIFY CANDIDATE CLASSES

The main domain entities are Playlist and Song.

STEP 2 — ASSIGN RESPONSIBILITIES

- Playlist stores its name and manages the collection of songs.
- Song stores song information and provides song behavior.

STEP 3 — CHOOSE THE RELATIONSHIP

Because a Song exists independently and may appear in several playlists, composition would make the wrong lifetime claim. A simple association, possibly implemented through references or pointers, expresses the requirement more safely.

STEP 4 — ADD ONLY USEFUL DETAIL

Show the attributes, operations, and multiplicities needed for the current design discussion. Leave function bodies to C++.

CHECKING A CLASS DIAGRAM

Before translating a diagram into code, read it as a set of design statements:

- Does every class represent a meaningful concept?
- Does each attribute belong to the class that owns that information?
- Does each operation belong to the class responsible for that behavior?
- Are data members private unless there is a strong reason otherwise?
- Can every multiplicity be explained in plain language?
- Do diamonds express real ownership and lifetime rules?
- Does every inheritance arrow pass the is-a test?

If you cannot explain a symbol as a sentence about the system, the diagram is probably more complicated than the design requires.

SUMMARY

- UML is a visual language for modeling and communicating software designs.
- A class diagram may show a class name, attributes, operations, and relationships.
- Attributes use the form name : type; operations add parameters and a return type.
- The symbols +, –, and # represent public, private, and protected visibility.
- Multiplicity expresses how many objects may participate in an association.
- Aggregation and composition both describe whole–part relationships, but composition expresses stronger lifetime ownership.
- Inheritance represents a genuine is-a relationship and points toward the base class.
- A class diagram is a blueprint; an object diagram is a snapshot.

UML is most useful when it makes a design decision easier to see, discuss, and translate into correct C++.